PERSONAL REFERENCE

CLI CHEAT SHEET

COMPLETE A-Z GUIDE

NixOS · Bash · River WM · yt-dlp · Partition Tools — March 2026

bash	nix	2026	100%
SHELL	NIXOS	EDITION	OFFICIAL

SHELL CMDS · NIXOS · PARTITIONS · RIVER WM

TABLE OF CONTENTS

SECTION 1	SHELL COMMANDS — eza, bat, head, tail, cp, mv, rm, sed, awk, fd...	1
SECTION 2	STORAGE & DRIVES — lsblk, udisksctl, mount, chmod, blkid, fsck	2
SECTION 3	BOOTABLE DRIVES & PARTITION MANAGEMENT TOOLKIT	3
SECTION 4	yt-dlp — Video, Playlist, Audio Downloads	4
SECTION 5	STEGHIDE — Hide Secrets in Images & Audio	5
SECTION 6	ARCH LINUX — WiFi, mkinitcpio, Kernel Panic, Fixes	6
SECTION 7	NIXOS + FLAKES + HOME MANAGER — Daily Cheatsheet	7
SECTION 8	RIVER WM — Keybinds Reference	8
SECTION 9	WORTH LOOKING AT — Official Resources & Links	9

Commands 101

Shell Commands

eza

```
$ eza                          # Basic list
$ eza -la                      # All files, long format
$ eza -lah                     # All files, long, human sizes
$ eza -T                       # Tree view
$ eza -T -L 2                  # Tree view, 2 levels deep
$ eza -lah --sort=modified     # Sort by modification time
$ eza -lah --sort=size         # Sort by size
```

less & more

```
$ less filename                # View with scrolling (q to quit)
$ more filename                # Simple pager
```

bat

```
$ bat filename                 # View with syntax highlighting
$ bat -p filename              # Plain output (no decorations)
$ bat -A filename              # Show all characters (spaces, tabs)
```

head & tail

First/last lines

```
$ head filename                # First 10 lines
$ head -n 20 filename          # First 20 lines
$ tail filename                # Last 10 lines
$ tail -n 20 filename          # Last 20 lines
```

file stuff

```
$ file filename                # Determine file type
$ stat filename                # Detailed file stats
$ du -h filename               # File/directory size human-readable
$ du -sh directory             # Total size of directory
```

touch, seq and echo

```
$ touch file1 file2 file3          Create multiple files
$ seq 7 | xargs -I {} touch {}.txt  Create multiple files at once
$ echo "content" > newfile         Create file with content
$ echo "more" >> existingfile      Append to file
```

mkdir

```
$ mkdir -p path/to/nested/dir      Create nested directories
$ mkdir dir1 dir2 dir3             Create multiple directories
$ mkdir -p project/{src,docs,tests}   Create multiple subdirs
```

cp

```
Copy files
$ cp source.txt dest.txt           Copy file
$ cp file1 file2 /dest/dir/        Copy multiple files to directory
$ cp -i source dest                Interactive (ask before overwrite)
$ cp -v source dest                Verbose (show what's being copied)
$ cp -p source dest                Preserve attributes (timestamp, permissions)

$ cp -r sourcedir/ destdir/        Copy directory recursively
$ cp -rv sourcedir/ destdir/       Copy directory with verbose output
$ cp -rp sourcedir/ destdir/       Copy preserving attributes
$ cp -ru sourcedir/ destdir/       Copy only newer files (update)


$ cp *.txt /dest/                  Copy all .txt files
$ cp file{1,2,3}.txt /dest/        Copy specific files
$ cp -r dir/{subdir1,subdir2} /dest/   Copy specific subdirectories
```

mv

```
Move/rename files
$ mv oldname newname               Rename file
$ mv source.txt /dest/dir/         Move file to directory
$ mv file1 file2 /dest/            Move multiple files
$ mv -i source dest                Interactive (ask before overwrite)
$ mv -v source dest                Verbose output


$ mv olddir newdir                 Rename directory
$ mv sourcedir /dest/path/         Move directory
```

rm

```
$ rm file1 file2 file3             Delete multiple files
$ rm -i filename                   Interactive (ask for each file)
$ rm -v filename                   Verbose output
$ rm -f filename                   Force delete (no confirmation)


$ rm -r dirname                    Remove directory recursively
```

```
$ rm -rf dirname                    Force remove directory (DANGEROUS)
$ rm -riv dirname                   Interactive verbose recursive removal
```

lsblik, udisksctl, mount, unmount

```
lsblk -f                          Show filesystem types
lsblk -o NAME,SIZE,TYPE,MOUNTPOINT   Custom columns

With your udisks2/udiskie setup
udisksctl status                  Show device status
udisksctl dump                    Detailed device info

Mount external drive manually (if not auto-mounted)
udisksctl mount -b /dev/sdX1      Mount device (replace sdX1)
udisksctl unmount -b /dev/sdX1    Unmount device

Traditional mount (requires sudo)
sudo mount /dev/sdX1 /mnt/usb     Mount to /mnt/usb
sudo umount /mnt/usb              Unmount

Check if udiskie auto-mounted
findmnt                           Show all mount points
mount | grep sdX                  Show specific device mounts

Copy to/from external drives
cp -rv /path/to/source /run/media/$USER/DRIVENAME/        Copy to USB
cp -rv /run/media/$USER/DRIVENAME/file ~/                 Copy from USB

Safe eject (before physical removal)
udisksctl unmount -b /dev/sdX1    Unmount first
udisksctl power-off -b /dev/sdX   Power off the device
```

chmod, chown

```
Change permissions
chmod 644 file                    rw-r--r-- (owner read/write, others read)
chmod 755 file                    rwxr-xr-x (owner all, others read/execute)
chmod 600 file                    rw------- (owner only)
chmod +x script.sh                Add execute permission
chmod -x file                     Remove execute permission
chmod u+x file                    Add execute for user/owner
chmod g+w file                    Add write for group
chmod o-r file                    Remove read for others
chmod -R 755 directory/           Recursive permission change

Change ownership (requires sudo)
sudo chown user:group file        Change owner and group
```

```
sudo chown user file              Change owner only
sudo chown :group file            Change group only
sudo chown -R user:group dir/     Recursive ownership change
```

shortcuts for bash shell

```
Keyboard shortcuts in bash (already available)
Ctrl+A                            Move to beginning of line
Ctrl+E                            Move to end of line
Ctrl+U                            Delete from cursor to beginning
Ctrl+K                            Delete from cursor to end
Ctrl+W                            Delete word before cursor
Ctrl+Z                            Suspend current process

Brace expansion
mkdir {dir1,dir2,dir3}            Create multiple directories
cp file.txt{,.bak}                Quick backup (copies to file.txt.bak)
mv file.{txt,md}                  Rename extension
echo {1..10}                      Generate sequence
echo {a..z}                       Generate alphabet
```

stdin, stdout & pipes

```
Redirection
command > file                    Redirect stdout to file (overwrite)
command >> file                   Redirect stdout to file (append)
command 2> file                   Redirect stderr to file
command &> file                   Redirect both stdout and stderr
command 2>&1                      Redirect stderr to stdout
```

pipes

```
command1 | command2               Pipe output to another command
cat file | sort | uniq            Chain multiple commands
```

...

```
find . -type f | wc -l            Count all files
du -h | sort -h | tail -10        10 largest items
ps aux | grep firefox | awk '{print $2}' | xargs kill   Kill all firefox
```

sed

```
In-place text replacement (sed)
sed -i 's/old/new/g' file.txt            Replace all occurrences
```

```
sed -i 's/old/new/' file.txt            Replace first occurrence per line
sed -i '10d' file.txt                   Delete line 10
sed -i '1,5d' file.txt                  Delete lines 1-5
sed -i '/pattern/d' file.txt            Delete lines matching pattern
sed -i 's/^/prefix/' file.txt           Add prefix to each line
sed -i 's/$/suffix/' file.txt           Add suffix to each line

Multiple files
sed -i 's/old/new/g' *.txt

With backup
sed -i.bak 's/old/new/g' file.txt       Creates file.txt.bak
```

awk

```
awk '{print $1}' file.txt               Print first column
awk '{print $1, $3}' file.txt           Print columns 1 and 3
awk -F':' '{print $1}' /etc/passwd      Custom delimiter
awk 'NR==10' file.txt                   Print line 10
awk 'NR>=10 && NR<=20' file.txt         Print lines 10-20
awk '/pattern/' file.txt                Print lines matching pattern
awk '{sum+=$1} END {print sum}' file    Sum first column
```

sort

```
sort file.txt                           Alphabetical sort
sort -r file.txt                        Reverse sort
sort -n file.txt                        Numerical sort
sort -h file.txt                        Human-readable numbers (1K, 1M)
sort -u file.txt                        Sort and remove duplicates
sort -k2 file.txt                       Sort by second column
sort -t':' -k3 -n /etc/passwd           Sort by 3rd field numerically
```

uniq

```
uniq file.txt                           Remove duplicate lines
uniq -c file.txt                        Count occurrences
uniq -d file.txt                        Show only duplicates
uniq -u file.txt                        Show only unique lines
```

wc

```
wc file.txt                             Lines, words, bytes
wc -l file.txt                          Count lines only
wc -w file.txt                          Count words only
wc -c file.txt                          Count bytes
wc -m file.txt                          Count characters
```

fd, find

```
fd (modern find replacement) - you NEED this
fd pattern                        Simple search
fd pattern /path                  Search in specific path
fd -H pattern                     Include hidden files
fd -I pattern                     Include ignored files (.gitignore)
fd -e txt                         Find by extension
fd -e txt -e md                   Multiple extensions
fd -t f pattern                   Files only
fd -t d pattern                   Directories only
fd -t l pattern                   Symlinks only
fd -s pattern                     Case-sensitive
fd -S 100M                        Files larger than 100MB
fd -S -100M                       Files smaller than 100MB
fd -x rm                          Execute command on results
fd pattern -x bat                 View all matching files

Find and execute with confirmation
fd pattern -x sh -c 'echo "Process $1?" && read -r && command "$1"' sh

Find modified in last N days
fd --changed-within 7d            Last 7 days
fd --changed-before 30d           More than 30 days ago

Find by user
find . -user $USER                Files owned by you
find . -type f -user $USER -exec ls -lh {} \;
```

split & csplit

```
Split large files
split -b 100M largefile.zip part_        Split into 100MB chunks
split -l 1000 largefile.txt part_        Split by 1000 lines per file
split -n 5 largefile.dat part_           Split into 5 equal parts

Merge split files back
cat part_* > original_file

Split with better naming
split -b 100M -d --additional-suffix=.zip largefile.zip part_

csplit - split by pattern
csplit file.txt '/pattern/' '{*}'        Split at each pattern occurrence
```

column, paste & join +++

```
column - columnate lists
column -t file.txt                      Auto-format into columns
mount | column -t                       Pretty-print mount info
cat /etc/passwd | column -t -s ':'      Format with custom separator

paste - merge lines
paste file1.txt file2.txt               Merge files side by side
paste -d ',' file1.txt file2.txt        Merge with comma delimiter
paste -s file.txt                       Merge all lines into one

join - merge files on common field
join file1.txt file2.txt                Join on first field
join -t ':' -1 3 -2 1 file1 file2       Join on specific fields

tee - write to multiple outputs
command | tee output.txt                Display and save to file
command | tee -a output.txt             Append to file
command | tee file1.txt file2.txt       Save to multiple files

xargs - build commands from input
find . -name "*.tmp" | xargs rm         Delete all .tmp files
fd -e txt | xargs -I {} cp {} backup/   Copy all txt files
echo "file1 file2 file3" | xargs -n 1 echo   Process one argument at a time
cat urls.txt | xargs -n 1 -P 4 wget     Download 4 URLs in parallel
```

find

```
findmnt                                 All mount points
`findmnt /dev/sda1                       Specific device`
findmnt -t ext4                         By filesystem type
findmnt -S /dev/sda1                    By source`
```

blkid

```
sudo blkid                              All block devices
sudo blkid /dev/sda1                    Specific device`
sudo blkid -s UUID /dev/sda1            Show only UUID`
```

fsck

```
sudo fsck /dev/sdX1                     Check and repair (UNMOUNT FIRST)`
sudo fsck -n /dev/sdX1                  Check without changes (safe)`
```

badblocks

```
sudo badblocks -v /dev/sdX              Read-only test`
sudo badblocks -nvs /dev/sdX            Non-destructive write test`
```

Create bootable drive

```
$ lsblk (make sure the targeted usb drive is present/mounted) \\$ sudo dd
if=/home/az/Desktop/artix-xfce-runit-20250407-x86_64.iso of=/dev/sda bs=4M
status=progress oflag=sync \\$ sync`

`\\$ free -h`
`\\$ df -h`

 Remove .trsh1000 folders items from your USB drive---
`replace "YOUR_USB_NAME" with your actual USB name, to know exactly run the command ---`

`\\$ ls /run/media/\\$USER/`
`\$ sudo rm -rf /run/media/\$USER/YOUR_USB_NAME/.Trash-1000/*`

`\\$ lsblk -o NAME,SIZE,FSTYPE,MOUNTPOINT | grep -v '\^loop'`

 usdisks2 + cryptsetup

drives automount stuff ---
`\\$ groups`
`\\$ sudo usermod -aG storage \\$USER`
`\\$ groups`
`\\$ swaymsg exit`
`\\$ groups`
`\\$ cd /run/media/\\$USER/`
for unmount -
`\\$ udisksctl unmount -b /dev/sda1`

note: first check the targeted USB drive by running the command lsblk ---

 Rename (label) an exFAT drive
`\\$ sudo exfatlabel /dev/sdX1 NewName`

 Rename (label) an NTFS drive
`\\$ sudo ntfslabel /dev/sdX1 NewName`

 Only Apply LUKS Encryption (no format)
`\\$ sudo cryptsetup luksFormat /dev/sdX1 && \ sudo cryptsetup open /dev/sdX1`
`secure_usb`

 Only Format as exFAT (no encryption)
`Way  1 Unmount the correct mountpoint sudo umount /run/media/zzz/mysam`

 Close the LUKS mapper
`\\$ sudo cryptsetup close luks-d13f7771-6d20-4707-aa85-ec55188e239a`

 Force kernel + udev to refresh before
- formatting
`\\$ sudo udevadm settle sudo partprobe /dev/sda`
```

```
- Verify closure
`\\$ lsblk -fp`

 Format safely as exFAT
`\\$ sudo mkfs.exfat -n MyB /dev/sda1`

`Way -2 Unmount the correct mountpoint sudo umount /run/media/zzz/mysam`
- Close the LUKS mapper
`\\$ sudo cryptsetup close luks-d13f7771-6d20-4707-aa85-ec55188e239a`

- Verify closure (mapper should disappear)
`\\$ lsblk -fp`

- Now it's safe to format as exFAT
`\\$ sudo mkfs.exfat -n MyB /dev/sda1`

- Only Format as NTFS (no encryption)
`\\$ sudo mkfs.ntfs -f -L MyUSB /dev/sdX1`

note: while formating + luks is being attempted, if anything it says like -
`Cannot exclusively open /dev/sda1, device in use - then simply run these`
command below ---
`\\$ lsblk-fp`

indetify your drive and see if mounted and showing its capacity, is showing
then
`\\$ sudo umount /run/media/zzz/FD86-090D (your drive name) then, \\$ lsblk -fp`
now it should not show the drive's capacity or like that & now it's all okay
to go on...

formatting a USB drive
- "exFAT + LUKS"
`\\$ sudo cryptsetup luksFormat /dev/sdX1 && sudo cryptsetup open /dev/sdX1`
`secure_usb && sudo mkfs.exfat -n mydrivesam /dev/mapper/secure_usb && sudo`
`cryptsetup close secure_usb`

formatting a USB drive
- "NTFS + LUKS"
`\\$ sudo cryptsetup luksFormat /dev/sdX1 && sudo cryptsetup open /dev/sdX1`
`secure_usb && sudo mkfs.ntfs -f -L Mydrivesand /dev/mapper/secure_usb &&`
`sudo cryptsetup close secure_usb`

 To unlock and mount the drive
- "exFAT"
`\$ sudo cryptsetup open /dev/sdX1 secure_usb && sudo mkdir -p /mnt/usb &&`
`sudo mount -t exfat /dev/mapper/secure_usb /mnt/usb`

 To unlock and mount the drive
- "NTFS"
`\$ sudo cryptsetup open /dev/sdX1 secure_usb && sudo mkdir -p /mnt/usb &&`
`sudo mount -t ntfs /dev/mapper/secure_usb /mnt/usb`
```

```
 To lock and mount --- "both exFAT & NTFS"
`\$ sudo umount /mnt/usb && sudo cryptsetup close secure_usb`

 PARTITION MANAGEMENT TOOLKIT —

 PART 1 — CREATE NEW PARTITIONS (WIPING OLD TABLE)
Identify your target drive
lsblk -fp
→ Example:
`/dev/sda 128G └─/dev/sda1 vfat MyOldUSB`
`/dev/sda = the *whole* drive. Double-check before continuing.`

Wipe old signatures (optional but safest)
`\\$ sudo wipefs --all /dev/sda`

`Removes any old partition tables/filesystem traces. Destroys all data —`
verify the device name first. Create new GPT partitions (modern layout)
`\\$ sudo parted /dev/sda --script mklabel gpt \ mkpart primary 0% 28GB \ mkpart`
primary 28GB 100%

#v(0.4cm)


 Creates two partitions:

`/dev/sda1 → 0–28 GB /dev/sda2 → 28 GB — end (~100 GB)`
`(For BIOS/MBR systems)`
`\\$ sudo parted /dev/sda --script mklabel msdos \ mkpart primary 0% 28GB \`
mkpart primary 28GB 100%

Same result using an MBR label. Verify -
`\\$ bash lsblk -fp`
Expected:
`/dev/sda ├─/dev/sda1 28G └─/dev/sda2 100G`

(Optional) Format
- exFAT:
`\\$ sudo mkfs.exfat -n Part1 /dev/sda1 sudo mkfs.exfat -n Part2 /dev/sda2`

- NTFS:
`\\$ sudo mkfs.ntfs -f -L Part1 /dev/sda1 sudo mkfs.ntfs -f -L Part2 /dev/sda2`

(Optional) Encrypt with LUKS
Example — 28 GB partition:
`\\$ sudo cryptsetup luksFormat /dev/sda1`
`\$ sudo cryptsetup open /dev/sda1 secure_usb1`
`\$ sudo mkfs.exfat -n MySecure1 /dev/mapper/secure_usb1`
`\$ sudo cryptsetup close secure_usb1`

Example — 100 GB partition:
```

```
`\\$ sudo cryptsetup luksFormat /dev/sda2`
`\$ sudo cryptsetup open /dev/sda2 secure_usb2`
`\$ sudo mkfs.exfat -n MySecure2 /dev/mapper/secure_usb2`
`\$ sudo cryptsetup close secure_usb2`

`CREATE NEW PARTITION WITHOUT LOSING EXISTING NTFS DATA*`
- (Non-destructive, NTFS-only; exFAT cannot be safely resized.)
`\\$ lsblk -fp`
`→ Find something like /dev/sda1 ntfs MyDrive.`
Unmount -
`\\$ sudo umount /dev/sda1`
`Check filesystem sudo ntfsfix /dev/sda1`

`Shrink filesystem to 100 GB sudo ntfsresize --size 100G /dev/sda1`
Shrinks only the filesystem, not the partition boundary.
Resize partition boundary to match
`\\$ sudo parted /dev/sda (parted) print (parted) resizepart 1 100GB (parted)`

quit

 Create new partition in free space (~28 GB)
`\\$ sudo parted /dev/sda --script mkpart primary 100GB 100%`

 Format the new partition
 exFAT:
`\\$ sudo mkfs.exfat -n NewPart /dev/sda2`

 NTFS:
`\\$ sudo mkfs.ntfs -f -L NewPart /dev/sda2`

Verify final layout
`\\$ lsblk -fp`
Expected:
`/dev/sda1 ntfs MyDrive /dev/sda2 exfat NewPart`

- FULL COMMAND SUMMARY
Identify lsblk -fp
If wiping & repartitioning
`sudo wipefs --all /dev/sda sudo parted /dev/sda --script mklabel gpt mkpart`
primary 0% 28GB mkpart primary 28GB 100%
If keeping data (NTFS shrink)
`sudo umount /dev/sda1 sudo ntfsfix /dev/sda1 sudo ntfsresize --size 100G`
`/dev/sda1 sudo parted /dev/sda resizepart 1 100GB sudo parted /dev/sda`
- script mkpart primary 100GB 100%

 Format new partition
`\\$ sudo mkfs.exfat -n NewPart /dev/sda2  or mkfs.ntfs -f -L NewPart /dev/sda2`

- Verify
lsblk -fp
]
```

yt-dlp

for the maximum best quality possible

```
$ yt-dlp --no-playlist --no-call-home --no-cache-dir --no-cookies -f
bestvideo+bestaudio/best --merge-output-format mkv --write-description --write-info-
json -P ~/Downloads -o "%(title).100s.%(ext)s" "VIDEO_URL"
```

for specific resolution/quality , say to say 1080p (feel free to just change the value of the video resolution/quality)-

```
$ yt-dlp --no-playlist --no-call-home --no-cache-dir --no-cookies -f
"bestvideo[height=1080]+bestaudio/best[height=1080]" --merge-output-format mkv --write-
description --write-info-json -P ~/Downloads -o "%(title).100s.%(ext)s" "VIDEO_URL"`
```

and in order to download an entire playlists videos also for the maximum best quality possible type this -

```
$ yt-dlp --no-call-home --no-cache-dir --no-cookies -f bestvideo+bestaudio/best --
merge-output-format mkv --write-description --write-info-json -P ~/Downloads -o
"%(playlist_title)s/%(playlist_index)03d - %(title).100s.%(ext)s" "PLAYLIST_URL"
```

and in order to download an entire playlists videos and for specific resolution/quality, say to say 1080p (feel free to just change the value of
`the video resolution/quality)-`

```
$ yt-dlp --no-call-home --no-cache-dir --no-cookies -f
"bestvideo[height=1080]+bestaudio/best[height=1080]" --merge-output-format mkv --write-
description --write-info-json -P ~/Downloads -o "%(playlist_title)s/
%(playlist_index)03d - %(title).100s.%(ext)s" "PLAYLIST_URL"
```

for downloading audio Single files type this (just replace wav, mp3 with flac for your targeted audio formats) -

```
$ yt-dlp --no-playlist --no-call-home --no-cache-dir --no-cookies -f bestaudio --
extract-audio --audio-format flac --audio-quality 0 --write-description --write-info-
json -P ~/Downloads -o "%(title).100s.%(ext)s" "VIDEO_URL"
```

for downloading audio playlists files type this (just replace wav, mp3 with flac for your targeted audio formats) -

```
$ yt-dlp --no-call-home --no-cache-dir --no-cookies -f bestaudio --extract-audio --
audio-format flac --audio-quality 0 --write-description --write-info-json -P ~/
```

```
Downloads -o "%(playlist_title)s/%(playlist_index)03d - %(title).100s.%(ext)s"
"PLAYLIST_URL"
```

Steghide

With Images

```
To embed a secret file in an Image file use this Command:
$ steghide embed -cf (image file's full name within extension) -ef
(secretfile, the file that wanna hide inside the image file)
```

To extract the secret file use this command:

```
$ steghide extract -sf (image file's full name within extension)
```

- With Audio

To embed a secret file in an Audio file use this Command:

```
$ steghide embed -cf (Audio file's full name within extension) -ef
(secretfile, the file that wanna hide inside the image file)
```

To extract the secret file use this command:

```
$ steghide extract -sf (Audio file's full name within extension)
```

Arch stuff + Fixes + Tweaking +

0.1. 📶 CONNECT WIFI

```
$ systemctl start iwd`
$ iwctl`
$ station list`
$ station wlan0 scan`
$ station wlan0 get-networks`
$ station wlan0 connect "SSID/Network/WiFi-name"`
```

and then put the WiFi password in and done and then run quit then run ping google.com to test if WiFi is working!

error at line 82 in /etc/mkinitcpio.conf indicates a syntax error - specifically an unclosed quote mark.

```
$ sudo nvim /etc/mkinitcpio.conf`
```

if there's a stray "o" or unmatched quote character at the end. Delete this entire line completely.

save, exit - then

```
$ sudo mkinitcpio -P`
```

Expected output should show:

```
==> Building image from preset: /etc/mkinitcpio.d/linux.preset: 'default'
==> Using default configuration file: '/etc/mkinitcpio.conf'
  -> -k /boot/vmlinuz-linux -g /boot/initramfs-linux.img
==> Starting build: '6.x.x-arch1-1'
==> Build complete.
```

Verify the file exists:

```
$ ls -lh /boot/initramfs-linux.img`
```

0.2. If You're Already in Kernel Panic

Boot from Arch installation media:

```
# Mount your root partition
mount /dev/sdXY /mnt  # Replace with YOUR partition
# Chroot into your system
arch-chroot /mnt
# Fix the config file
nano /etc/mkinitcpio.conf
# Delete the stray quote at the bottom
# Regenerate
mkinitcpio -P
# Verify
ls -l /boot/initramfs-linux.img
# Exit and reboot
exit
umount -R /mnt
reboot
```

0.3. Prevent This Forever

Lock the file after fixing:

```
$ sudo chattr +i /etc/mkinitcpio.conf`
```

This makes the file immutable. To edit in the future:

```
sudo chattr -i /etc/mkinitcpio.conf  # unlock
# edit
sudo chattr +i /etc/mkinitcpio.conf  # lock again
```

1. disable sshrootlogin

```
$ nvim /etc/ssh/sshd_config`
```

inside, look for a line named - "PermitRootLogin" (with a hash symbol before it)

there it may set to yes or prohibit-password.

just set it to no, and save and maybe reboot maybe!

NixOS + Flakes + Home Manager

Daily Cheatsheet

Edit system packages or config

```
$ sudo nvim /etc/nixos/configuration.nix
$ sudo nixos-rebuild switch --flake /etc/nixos#nixos
```

Edit dotfiles / programs (Home Manager)

```
$ sudo nvim /etc/nixos/home.nix
$ sudo nixos-rebuild switch --flake /etc/nixos#nixos
```

Monthly update (equivalent of pacman -Syu)

```
$ sudo nix flake update /etc/nixos
$ sudo nixos-rebuild switch --flake /etc/nixos#nixos
```

Rollback if something breaks

```
$ sudo nixos-rebuild switch --rollback
```

Garbage collect — free disk space

```
$ sudo nix-collect-garbage -d
$ sudo nixos-rebuild switch --flake /etc/nixos#nixos
```

Search packages

```
$ nix search nixpkgs packagename
```

Check generations (history)

```
$ sudo nix-env --list-generations --profile /nix/var/nix/profiles/system
```

Fresh Machine Restore

Clone and apply entire system from scratch

```
# 1. Install NixOS minimal from ISO
# 2. Generate fresh hardware config
$ sudo nixos-generate-config

# 3. Clone your config repo
$ git clone https://github.com/shaonahmedronok1/nixos-config

# 4. Copy your files (NOT hardware-configuration.nix)
$ sudo cp nixos-config/flake.nix /etc/nixos/
$ sudo cp nixos-config/flake.lock /etc/nixos/
$ sudo cp nixos-config/configuration.nix /etc/nixos/
$ sudo cp nixos-config/home.nix /etc/nixos/

# 5. One command — entire system materializes
$ sudo nixos-rebuild switch --flake /etc/nixos#nixos
```

Key Concepts

File structure

```
/etc/nixos/
├── flake.nix                  # entry point, versions pinned
├── flake.lock                 # exact version snapshot — commit this to git
├── configuration.nix          # system packages, services, hardware
├── home.nix                   # all dotfiles, user programs
└── hardware-configuration.nix # machine-specific — regenerate fresh on new PC
```

Rules — never break these

```
# Never edit ~/.config directly — Home Manager owns it via symlinks
# Never copy hardware-configuration.nix to a new machine — always regenerate
# Always commit flake.lock to GitHub after nix flake update
# home.stateVersion — never change this value ever
```

/.config explained

```
# ~/.config still exists but everything is symlinks pointing to /nix/store/
# Example:
~/.config/waybar/config.jsonc -> /nix/store/abc123.../waybar-config.jsonc
~/.config/river/init          -> /nix/store/abc123.../river-init
~/.bashrc                     -> /nix/store/abc123.../bashrc

# Edit home.nix → rebuild → symlinks update automatically
# Never touch the files directly
```

River WM Keybinds

```
Super + Return          Open terminal (alacritty)
Super + Space           App launcher (fuzzel)
Super + B               Chromium
Super + E               Nautilus file manager
Super + C               Close window
Super + F               Toggle fullscreen
Super + Shift + Space   Toggle float
Super + V               Clipboard history (cliphist)
Super + W               Cycle wallpaper
Super + Z               Screenshot
Super + Shift + X       Lock screen (swaylock)
Super + N               Dismiss notification
Super + Shift + N       Dismiss all notifications
Super + Tab             Previous tag
Super + Shift + E       Exit river

Super + H/J/K/L         Focus window left/down/up/right
Super + Shift + H/J/K/L Swap window left/down/up/right
Super + 1-9             Switch to tag 1-9
Super + Shift + 1-9     Move window to tag 1-9
Super + Control + 1-9   Toggle focused tags
Super + R               Enter resize mode

Super + Page_Up         Brightness up (ddcutil)
Super + Page_Down       Brightness down (ddcutil)
Super + F1              Mute audio
Super + F2              Volume down
Super + F3              Volume up
```

Worth looking at

https://linux.die.net

https://fmhy.net

https://www.commandlinefu.com/commands/browse/sort-by-votes

https://www.commandlinefu.com/commands/browse

https://www.gnu.org/software/coreutils/manual

https://wiki.archlinux.org/title/Core_utilities

https://wiki.archlinux.org/title/Bash

https://dotfiles.substack.com

https://github.com/mhoye/moderntools

https://github.com/natpen/awesome-wayland

https://github.com/toolleeo/awesome-cli-apps-in-a-csv?tab=readme-ov-filefile-handling

https://github.com/mhoye/moderntools

https://github.com/rothgar/awesome-tuis
https://github.com/jlevy/the-art-of-command-line
https://github.com/alebcay/awesome-shell
https://github.com/agarrharr/awesome-cli-apps
https://github.com/Jacksaur/Gorgeous-GRUB
https://github.com/toolleeo/awesome-cli-apps-in-a-csvfile-handling
https://github.com/k4m4/terminals-are-sexy
https://igwiki.lyci.de/wiki/GNU/Linux_ricing
https://igwiki.lyci.de/wiki/List_of_recommended_GNU/Linux_software
https://lukesmith.xyz/programs
https://harmful.cat-v.org/software
https://github.com/mayfrost/guides/blob/master/ALTERNATIVES.md
https://comfy.guide/client/luks
https://comfy.guide/client/gpg
https://distro.tube
https://wiki.archlinux.org/title/Main_page
https://stackoverflow.com/questions
https://unix.stackexchange.com
https://github.com/mayfrost/guides/blob/master/CHECKLIST.md
https://www.gnu.org/software/librejs
https://www.gnu.org/prep/ftp.html
https://www.gnu.org/manual/blurbs.html
https://nixos.org/manual/nixos/stable
https://nix-community.github.io/home-manager
https://wiki.nixos.org/wiki/Flakes
https://search.nixos.org/packages
https://nixos-and-flakes.thiscute.world
https://nix.dev
https://nixos.org/manual/nixos/stable
https://nix-community.org